

Artículo 1: Hardware y Bases de Datos (Processing-In-Memory)

- **Referencia:** Kim, J., et al. (2025). Breaking Through the Memory Wall of Transactional Database Management Systems Using Processing-In-Memory Hardware. *Proceedings of the VLDB Endowment*, 18, 4241-4254. [doi:10.14778/3749646.3749690]
- **Problema que aborda:** El límite de rendimiento en bases de datos en memoria (OLTP) dictado por el ancho de banda del canal de memoria que conecta la RAM con el CPU.
- **Propuesta de los autores:** Descargar componentes del sistema gestor directamente a hardware *Processing-in-Memory* (PIM), usando núcleos lógicos dentro de los propios chips DRAM para procesar datos *in situ*.
- **Resultado principal:** Se supera la "pared de memoria", logrando un aumento drástico en el procesamiento de transacciones al evitar transferir los datos físicamente hacia el procesador.
- **Relación con Unidad I:** Se relaciona con la **arquitectura del sistema de base de datos** (arquitectura de hardware y concurrencia).
- **Aporte al proyecto:** Me permitió visualizar cómo el diseño lógico de compuertas y la electrónica a bajo nivel impactan directamente el rendimiento algorítmico y el manejo de punteros en el desarrollo del software.

Artículo 2: Bases de Datos Multimodelo

- **Referencia:** Zhao, Y., et al. (2024). Towards a Unified Multi-Model Database Management System: Architecture and Optimization. *Proceedings of the ACM on Management of Data (SIGMOD)*, 2(1), 1-15. [doi: 10.1145/3829078]
- **Problema que aborda:** El surgimiento de aplicaciones modernas que requieren procesar simultáneamente datos relacionales, grafos y documentos (JSON), lo que obliga a las empresas a mantener múltiples gestores separados, generando redundancia e inconsistencia.
- **Propuesta de los autores:** Una arquitectura de sistema gestor multimodelo que utiliza un núcleo de almacenamiento unificado y un único lenguaje de consultas capaz de transicionar entre esquemas rígidos y flexibles dinámicamente.
- **Resultado principal:** Se reduce drásticamente la redundancia de datos y los tiempos de latencia en la transferencia entre sistemas aislados, simplificando la administración.
- **Relación con Unidad I:** Se relaciona estrechamente con la clasificación de los **modelos de datos** y la superación de la **redundancia e inconsistencia** (archivos frente a bases de datos).
- **Aporte al proyecto:** Demuestra cómo abstraer múltiples estructuras de datos matemáticos bajo una sola interfaz lógica, lo cual simplificaría el diseño backend de cualquier sistema complejo sin saturar la infraestructura.

Artículo 3 : Inteligencia Artificial en el Gestor de Base de Datos

Referencia: Marcus, R., et al. (2023). Deep Reinforcement Learning for Query Optimization in Relational Databases. *Proceedings of the VLDB Endowment*, 16(8), 1820-1833. [doi: 10.14778/3342263.3342644]

- **Problema que aborda:** Los optimizadores de consultas tradicionales de los gestores relacionales usan reglas estáticas y heurísticas matemáticas que fallan o son ineficientes ante cargas de trabajo dinámicas y altamente complejas.
- **Propuesta de los autores:** Sustituir el módulo clásico del optimizador de consultas por un agente de aprendizaje por refuerzo profundo (Deep RL) que aprende empíricamente el mejor plan de ejecución (árboles de búsqueda) interactuando con los datos.
- **Resultado principal:** El gestor aprende a generar planes de ejecución de consultas mucho más rápidos de forma autónoma, reduciendo la necesidad de que el Administrador de Base de Datos (DBA) ajuste los índices manualmente.
- **Relación con Unidad I:** Se vincula directamente con los **módulos componentes de un sistema gestor**.
- **Aporte al proyecto:** Resulta inspirador para el análisis algorítmico, ya que muestra cómo reemplazar un componente lógico tradicional (heurística) por uno adaptativo en C++ o lenguajes de alto rendimiento, optimizando el consumo de recursos.

Los tres artículos analizados (Kim et al., Zhao et al., y Marcus et al.) abordan el mismo desafío fundamental: los cuellos de botella de la arquitectura clásica de bases de datos frente a las demandas de rendimiento contemporáneas. Ya sea lidiando con el volumen masivo de transacciones, la variedad de los datos o la complejidad de las consultas, los tres proponen rediseñar o reemplazar componentes históricos del Sistema Gestor de Bases de Datos (SGBD) para alcanzar una mayor eficiencia y escalabilidad.

La diferencia central radica en la capa del sistema que intentan revolucionar. El primer artículo (Kim et al.) interviene en la **capa física y de hardware**, acercando el procesamiento directamente a los chips de memoria (PIM) para evitar el traslado de datos. El segundo artículo (Zhao et al.) actúa en la **capa estructural**, proponiendo modelos multimodelo para unificar paradigmas (relacional, grafo, documento) y evitar la redundancia. El tercer artículo (Marcus et al.) modifica la **capa lógica de software**, integrando inteligencia artificial directamente en los módulos internos del SGBD para optimizar la toma de decisiones algorítmicas sin intervención humana.

A partir de estas lecturas, surge un problema abierto crítico: la **pérdida de predictibilidad e independencia**. Al integrar arquitecturas en memoria profunda, motores multimodelo dinámicos y optimizadores basados en redes neuronales, el SGBD se convierte en una "caja negra". Esto dificulta garantizar características fundamentales de las bases de datos como la recuperación determinista ante fallos y la independencia absoluta de los datos, ya que el comportamiento del sistema cambia dinámicamente y la depuración de errores se vuelve un desafío de ingeniería sumamente complejo.